

Rapport Technique Mensuel Complet : Mars 2026

Refonte Microscopique, Optimisation des Performances et Migration Multiplateforme GPU de LULESH 2.0

Période du projet : 02/03/2026 ~ 29/03/2026

Positionnement du projet : Modernisation de l'application mandataire HPC LULESH 2.0 et portage vers Kokkos

Plateformes de test : macOS Apple Silicon (4 cœurs P + 6 cœurs E, validation OpenMP) / Linux + NVIDIA GPU (plateforme cible CUDA)

Référence de justesse : Strictement maintenue tout au long du mois avec l'échelle

OMP_PROC_BIND=close ./lulesh2.0 -s 10 -i 50 produisant Final Origin Energy = 8.104796e+04

I. Découplage de l'architecture héritée et nettoyage approfondi en C++20

Au cours de la première semaine de ce mois, l'équipe a procédé à un découpage chirurgical des responsabilités physiques du fichier unique lulesh.cc de 2813 lignes, qui mélangeait les directives MPI et OpenMP.

1. Indépendance de 10 modules physiques : Le code a été découpé avec précision en

lulesh-geometry (fonctions de forme et caractéristiques géométriques),

lulesh-stress (intégration des contraintes et contrôle du sablier),

lulesh-nodal (cinématique et mise à jour de la mécanique),

lulesh-kinematics (gradient de vitesse et taux de déformation),

lulesh-viscosity (viscosité artificielle),

lulesh-eos (équation d'état),

lulesh-timestep (contraintes de pas de temps),

lulesh-integration (boucle principale Leap-Frog),

lulesh-init (initialisation du maillage)

et lulesh-util (analyse en ligne de commande et sorties),

plus une entrée main indépendante.

2. Réorganisation des variables du domaine (Domain) : Les quelque 80 variables membres privées de la classe Domain d'origine ont été regroupées de force selon leur sémantique en trois structures :

NodeArrays (gestion des coordonnées et vitesses des nœuds, etc.),

ElemConnectivity (gestion de la topologie et de la connectivité des éléments)

et ElemState (gestion de l'état physique des éléments).

3. Élimination des pointeurs et modernisation de la syntaxe : Les anciens modèles de gestion de mémoire par pointeurs nus Allocate<T> et Release<T> ont été totalement abandonnés, remplacés par std::vector<T> pour exploiter le mécanisme RAII. Dans l'ancien code, NULL a été remplacé par nullptr, typedef Real_t double par l'alias using, et les définitions de macros telles que #define FREEX 0x1 ont été mises à niveau vers constexpr Int_t. Les macros d'encapsulation personnalisées MAX et SQRT ont été entièrement supprimées au profit des appels directs à <cmath> et std::max.

4. Nettoyage au niveau de l'ingénierie : Suppression de 8 inclusions d'en-têtes standard C inutiles telles que <limits>, nettoyage des commentaires de débogage obsolètes, remplacement de toutes les gardes d'inclusion #ifndef par le moderne #pragma once, et suppression de la macro d'alignement obsolète CACHE_COHERENCE_PAD_REAL. La structure des répertoires du

projet a été normalisée en apps/, scripts/, samples/ et tests/baselines/, et le système de build a été entièrement mis à niveau vers CMake 3.16+.

II. Changements de stratégie parallèle de base (OpenMP → Kokkos)

Lors du remplacement des #pragma omp, deux modifications de mécanisme majeures ont été apportées pour contourner les limitations de bas niveau :

Optimisation de la capture de fermeture (Closure Capture) : Abandon de la capture par valeur KOKKOS_LAMBDA ([=]), au profit de la capture par référence [&] combinée à l'extraction des pointeurs .data() des std::vector (contournant ainsi les restrictions const de la plateforme CPU).

Évolution du mécanisme de réduction : Tableau manuel de valeurs minimales locales → Kokkos::MinLoc → Réduction variadique pure Kokkos::Min.

En combinant plusieurs petits noyaux (Kernels) indépendants en grands noyaux, nous avons considérablement éliminé les coûteuses barrières de synchronisation des threads (Barriers).

Module cible / Logique	État avant fusion	État après fusion	Gain principal
Contrôle du sablier (Stress)	Fonctions indépendantes CalcFBHourglass... etc.	Intégré en ligne en 1 noyau	Réduction des barrières, économie de ~35 Mo de mémoire pour vecteurs temporaires
Phase de préparation EvalEOS	5 noyaux (Collecte et compression de données dispersées)	1 noyau	Élimination des lancements de noyaux et synchronisations superflus
Super CalcEnergy	11 noyaux (Phases e1/q1/e2/e3/q2 + 3 boucles de pression)	1 noyau (Suppression directe des fonctions obsolètes)	Refonte la plus critique, compression extrême du nombre de barrières
Cinématique des nœuds	Mise à jour vitesse, mise à jour position (2 op. indépendantes)	1 noyau	Réduction des barrières
Réduction du pas de temps	Réduction Courant, Réduction Hydro (2 réductions indép.)	1 noyau à double réduction	Réduction du coût de synchronisation

Module cible / Logique	État avant fusion	État après fusion	Gain principal
Module EOS	Calcul vitesse du son, Dispersion des données (Scatter)	1 noyau	Réduction des barrières
Propriétés des matériaux	Troncature max, troncature min, vérification volume (3 noyaux)	1 noyau	3 en 1, logique simplifiée
Contrainte initiale	Fonction InitStressTerms..., calcul Scatter	1 noyau (Calcul en ligne intégré au Scatter)	Réduction des barrières
Gradient cinématique	Calcul du gradient, MàJ des éléments de Lagrange	1 noyau	Variable D conservée sur la pile, économie de bande passante VRAM

Ces 9 fusions ont apporté un bond de performance spectaculaire :

Indicateur de performance	Avant opt. (OpenMP d'origine)	Après opt. (Version Kokkos fusionnée)	Amélioration
Coût de synchronisation des threads	Niveau de référence	Réduction de ~27 barrières / itération	Élimination significative des temps d'attente CPU
Débit FOM	2572	5518	+114% (Plus que doublé)
Temps d'exécution de référence	7.10 secondes	3.30 secondes	Accélération de 2.15×

III. Diagnostic de l'ordonnancement des cœurs hétérogènes P/E et compression de la mémoire de bas niveau

Lors des tests sur macOS, trois goulots d'étranglement majeurs liés au matériel ont été identifiés et corrigés :

Ordonnancement des cœurs P/E (Chute de performance de 22%) : Passer de 4 threads (cœurs P rapides) à 5 threads (incluant 1 cœur E lent) faisait paradoxalement grimper le temps de calcul de 1.25s à 1.53s. Cause : Les barrières globales de Kokkos forcent les cœurs rapides à s'arrêter pour attendre le cœur lent à chaque fin de boucle. Solution : Injection de la commande `sysctl hw.perflevel0.physicalcpu` dans le script de lancement pour détecter automatiquement et lier l'exécution uniquement aux cœurs P.

Élimination du verrou global malloc (154 000 allocations) : La déclaration dynamique de 14 tableaux `std::vector` à chaque itération du module EOS générerait des centaines de milliers d'allocations mémoire (heap), paralysant les threads à cause des verrous globaux (`glibc/libmalloc`). Solution : Création d'une structure `EosTemps` pour pré-allouer tout l'espace nécessaire en une seule fois lors de l'initialisation du système.

Coup de grâce au faux partage de cache (False Sharing) : L'alignement par défaut sur 8 octets du tampon `fx_elem` faisait que les données traitées par différents cœurs tombaient sur la même Ligne de Cache CPU, provoquant une tempête d'invalidations d'écritures. Solution : Utilisation de `std::aligned_alloc(64, ...)`. Comme chaque élément fait exactement 64 octets (8 doubles), il monopolise désormais sa propre ligne de cache, isolant physiquement les accès.

Aplatissement des boucles et Résultat final : En complément, 11 boucles indépendantes basées sur les régions ont été "aplaties" en une seule boucle globale, économisant 20 barrières supplémentaires par pas de temps. Grâce à ces optimisations, le temps sur 4 cœurs a été compressé à 1.10s, portant le ratio d'accélération final à $2.51\times$.

IV. Restructuration de l'infrastructure GPU et de la fermeture des fonctions de noyau

Pour repousser le plafond de puissance de calcul du CPU, la seconde moitié du mois s'est concentrée sur la compilation et l'exécution natives de ce code extrêmement optimisé sur l'architecture CUDA des GPU NVIDIA.

Pénétration de l'infrastructure côté périphérique (Phase 1) :

- 1. Injection de visibilité des fonctions** : La macro `KOKKOS_INLINE_FUNCTION` a été ajoutée une par une aux 8 fonctions auxiliaires géométriques de bas niveau dans `lulesh-geometry.cc` (y compris le calcul des dérivées des fonctions de forme, des normales des nœuds, des dérivées de volume, de la longueur caractéristique, de la surface des faces, etc.), permettant à ces fonctions de générer simultanément des symboles côté hôte (host) et côté périphérique (device) lors de la compilation.
- 2. Transformation des structures de mémoire en View** : Les 14 vecteurs temporaires dans `EosTemps`, le tableau `vnew` du module d'intégration temporelle, ainsi que le déterminant de volume `determ` et le tampon de dispersion à alignement forcé du module de contrainte, ont tous été détruits et reconstruits sous forme de `Kokkos::View<Real_t*>` dotés de capacités de gestion automatique de l'espace mémoire.
- 3. Rendre les portées publiques** : Pour permettre aux Lambdas de capturer des données au-delà des limites de l'hôte et du périphérique, les trois grandes structures imbriquées de la classe

Domain ont été déplacées de force de la zone privée à la zone publique.

Refonte du paradigme de capture des fermetures côté périphérique (Phase 2) :

Refonte complète de plus de 20 fonctions de noyau réparties dans 6 modules pour les rendre compatibles avec le GPU :

Aspect Technique	Avant (Spécifique CPU)	Après (Compatible GPU NVIDIA)	Raison de la modification
Mode de Capture	Capture par référence [&] sur la pile de l'hôte	Variables locales Kokkos::View + capture par valeur KOKKOS_LAMBDA ([=])	Permet d'envoyer les poignées de métadonnées légères directement dans la mémoire vidéo (VRAM).
Instructions Illégales	fprintf(stderr, ...), exit() std::sqrt, std::fabs	Kokkos::abort Kokkos::sqrt, Kokkos::fabs	Le GPU ne prend pas en charge les appels système d'E/S standard de la bibliothèque C/C++.
Dépendances Hôte	Appel à la fonction hôte CollectDomainNodes...	Boucle de collecte native intégrée (adressage à 8 nœuds)	Le GPU ne peut pas planifier ni exécuter directement une fonction résidant sur l'hôte.
Logique CPU exclusive	Branche d'écriture directe (optimisée monothread)	Utilisation obligatoire du mode sécurisé Scatter/Gather	L'écriture directe provoquerait des conditions de course (Race Conditions) catastrophiques avec la concurrence massive du GPU.

V. Déminage complet de la chaîne CUDA et bataille contre les dépassements de mémoire

Lors de la phase finale de compilation croisée sous Linux et de déploiement sur la machine physique, nous avons été confrontés à une résistance féroce à bas niveau, qui a finalement été entièrement surmontée après une investigation rigoureuse.

Réparation de la chaîne de compilation et d'édition de liens :

1. Impasse de violation de la règle ODR : L'ajout de **KOKKOS_INLINE_FUNCTION** a introduit inline, ce qui a empêché l'éditeur de liens de trouver les entités en ligne des fonctions géométriques dans les fichiers objets en raison de la règle de définition unique (ODR) de C++.

Nous avons résolument vidé le fichier source *lulesh-geometry.cc* et remonté de force toutes les implémentations en ligne dans le fichier d'en-tête *include/lulesh-geometry.h*.

2. Pollution de l'espace de noms Lambda : CMake a marqué à tort les attributs des fichiers sources comme *LANGUAGE CUDA*, ce qui a amené le compilateur sous-jacent *nvcc* à contourner le wrapper Kokkos pour analyser directement les fichiers C++, injectant de force un préfixe illégal *_INTERNAL_* aux expressions Lambda, ce qui a fait échouer la déduction des modèles. La solution a consisté à supprimer cet attribut et à forcer le routage via *COMPILE_LANGUAGE:CXX* vers *nvcc_wrapper* pour une interception unifiée.

3. Défaut critique de CUDA_LAMBDA : L'enquête a finalement révélé que lors de la compilation de la bibliothèque sous-jacente Kokkos sur la machine physique Linux cible, l'option fatale *Kokkos_ENABLE_CUDA_LAMBDA=ON* avait été omise de manière inattendue, ce qui a provoqué la dégradation de toutes les macros en de simples *[=]* sans attribut de périphérique. Nous avons immédiatement écrit le script de reconstruction en un clic *build-cuda.sh* pour refondre toutes les dépendances sous-jacentes.

Défense contre trois types de plantages transfrontaliers CudaSpace :

Plantage A (Module EOS) : L'utilisation directe d'une View non gérée pour envelopper le pointeur brut de l'hôte *regElemRaw* a conduit le cœur du GPU à tenter de lire sans autorisation la mémoire de l'hôte. Nous avons introduit un mécanisme d'allocation rigoureux, en déclarant d'abord un miroir *HostSpace*, puis en exécutant une copie *Kokkos::deep_copy* pour le pousser vers la mémoire vidéo.

Plantage B (Module Init global) : Une fois que le système avait alloué les View du périphérique pendant la phase de construction, il exécutait directement une opération d'assignation à zéro scalaire dans une boucle for côté hôte. Nous avons activé la surcharge scalaire de *Kokkos::deep_copy* pour l'initialisation des constantes, et pour les logiques de construction d'hôte complexes comme la connectivité du maillage, nous avons activé *create_mirror_view* pour construire une mémoire fantôme, avant d'effectuer une copie inverse vers le GPU une fois l'orchestration de l'hôte terminée.

Plantage C (Module de viscosité Viscosity) : La logique d'origine, après l'exécution du noyau, utilisait une boucle for géante côté hôte pour lire un par un les résultats *domain.q(i)* dans la mémoire vidéo du GPU afin de déterminer si une interruption de limite avait été déclenchée. Cela présentait non seulement des risques de dépassement, mais provoquait également une latence de communication catastrophique sur le bus PCI-E. Nous l'avons réécrit en tant que réduction variadique *Kokkos::parallel_reduce* entièrement déployée au sein du GPU, ne renvoyant à l'hôte qu'un seul indicateur booléen final, scellant ainsi complètement le dernier point de fuite de données.

VI. Feuille de route des défis du mois prochain

Après avoir éliminé tous les obstacles liés à la compilation multiplateforme et à l'exécution, les tâches du mois d'avril viseront directement les limites de performance du calcul hétérogène et le déploiement en grappe (cluster) :

1. Analyse des valeurs extrêmes et étalonnage (Benchmarking) : Sur l'architecture Linux + NVIDIA, augmenter l'échelle de calcul au niveau *-s 45 -i 200*, et mesurer avec précision le débit FOM (zones/seconde) lorsque des milliers d'unités de calcul sont concurrentes, afin de vérifier le véritable ratio d'accélération matérielle du GPU.

2. Briser les contraintes de sérialisation de l'EOS (Introduction de CUDA Stream) :

Actuellement, les boucles basées sur les régions (Region) dans le code sont encore réparties en série sur le GPU en raison d'un déséquilibre de charge. Nous prévoyons de lancer un assaut spécifique sur la planification des **flux asynchrones CUDA Stream**, afin de réaliser un chevauchement concurrent des tâches de calcul multi-régions sur les multiprocesseurs de flux (SM), en extrayant complètement la puissance de calcul inutilisée sous des maillages à petite échelle.

3. Retour de l'infrastructure distribuée hétérogène : Réintégrer dans l'architecture modernisée l'infrastructure distribuée qui a été retirée lors de la refonte de la première semaine (calcul GhostIdx, décalage du domaine de coordonnées et mécanisme d'échange de frontières Halo à 6 faces, etc.). Il s'agira également d'évaluer de manière systématique s'il convient d'encapsuler les interfaces MPI natives via le traditionnel KokkosComm, ou d'introduire de manière radicale Kokkos Remote Spaces (modèle d'espace d'adressage global PGAS) pour construire la prochaine génération de base de communication transparente inter-nœuds.